

MagNET — Hardware Test Plan

A sequential walkthrough for validating each major firmware rev on the bench. Run top-to-bottom; stop at the first failed step and surface the symptom — every step's expected output is documented so a deviation is the diagnostic signal.

Active rev: **0.5.0-spore** (gen plumbing + lineage gate + Capsule Redis sidecar). **Last refreshed:** 2026-05-05.

How to update this file

When a major rev lands, update **in place** rather than appending — git history preserves the prior plan if you ever need it. Concretely:

1. Bump the *Active rev* line above to the new **MAGNET_GEN_STR**.
 2. Bump the *Last refreshed* date.
 3. For each new feature, add a numbered step in the most natural place (close to related features, not at the end).
 4. Remove steps that no longer reflect the codebase — a stale "verify X works" item is worse than no item.
 5. Keep every step's three-part shape: **Do, Expected, If it fails**. That shape is what makes the file usable cold.
 6. Update *Revision history* at the bottom with a one-line summary of what the rev added so you can `git log test-plan.md` to retrace decisions.
-

0. Prereqs

- **Devices on bench:** Dial (ruler), at least one peer (Echo or Matrix or C3U), Capsule (Redis variant), optional Camera.
- **Same Wi-Fi 2.4 GHz SSID** for everything; mDNS unblocked. macOS: System Settings → Privacy → Local Network must allow Terminal/Python.
- **Laptop tools:** `redis-cli` (`brew install redis`), `nc`, Python 3.11+ for `scripts/fake_ruler.py`.
- **Two terminals:** one running `pio device monitor -e <env>` for whichever device you're inspecting; one for laptop scripts and `redis-cli`.

1. Flash everything

Flash the **ruler first** so newer rulers can recognise older nodes during the transition (see `docs/MagNET-Generations.md`, Compatibility matrix).

```
cd M5StackDial-m5gfx-demo-ESPIDFORTH      && pio run -t upload
cd ../M5Capsule_Hive_Scribe_Redis          && pio run -t upload    # Redis
variant; replaces old Scribe
cd ../M5Atom_Echo_Hex_Hive_Test            && pio run -t upload
```

```
cd ../M5_Hive_Camera          && pio run -t upload  #
optional
cd ../M5Atom_Matrix_Hive_Test  && pio run -t upload  #
optional
```

Expected on each boot banner:

```
MagNET gen 0.5.0-spore
```

If it fails: the device wasn't rebuilt with `magnet_gen.h`. Force a fresh PIO build (`pio run -t fullclean`) and reflash.

2. Baseline hive — no gating, no Redis

Goal: confirm we haven't broken multi-node join.

Do:

1. Provision Wi-Fi on each device via BLE (nRF Connect → connect → write SSID + password → commit).
2. On the Dial USB serial, run `ruler-status`.

Expected:

```
ruler:  MagNET-ruler-XXXX
hive:   beehive-1
gen:    0.5.0-spore
gate:   lineage-auth=off
peers:  N
  [0] MagNET-biologic-aaaa  spawn  0.5.0-spore  12s ago
  [1] MagNET-biologic-bbbb  scribe 0.5.0-spore  8s ago
```

Every peer column shows `0.5.0-spore` — that confirms Layer 1 round-trip.

If it fails:

- A peer shows `(no-gen)` → that device wasn't reflashed with the gen-aware firmware.
- No peers connect → check Wi-Fi LED; `prov-status` on each peer should print a real IP, not `N/A`; check the macOS firewall and Local Network permission.

3. Verify Layer 1 — gen field log

Do: Watch any peer's serial monitor while it joins.

Expected on the Dial side:

```
WELCOME node=MagNET-biologic-XXXX role=spawn session=... gen=0.5.0-spore
```

If it fails: `gen=(none)` means the peer's HELLO didn't include gen — reflash that peer.

4. Verify Layer 2 positive path — lineage gate ON

Do: On the Dial USB serial:

```
1 lineage-auth
ruler-status
```

Expected:

```
lineage-auth: ON
...
gate: lineage-auth=ON
```

Do: Power-cycle a peer (Echo's reset button, or unplug/replug Capsule).

Expected on the Dial:

```
craw_hive_ruler: tx CHALLENGE to MagNET-biologic-XXXX lineage=spore
craw_hive_ruler: lineage auth OK MagNET-biologic-XXXX lineage=spore
craw_hive_ruler: WELCOME node=... role=... session=... gen=0.5.0-spore
```

Expected on the peer:

```
craw_hive_node: tx RESPONSE for lineage challenge
craw_hive_node: state=3 (...) # CRAW_HIVE_NODE_JOINED
```

The peer should rejoin within ~2 s.

If it fails: loops on `BACKOFF (lineage_auth)` → check that `MAGNET_LINEAGES[]` in `components/craw_hive/magnet_lineages.c` was identical at compile time on both sides (a typo in the key bytes will break this silently).

Cleanup before continuing:

```
0 lineage-auth
```

5. Verify Layer 2 negative path — `fake_ruler.py` only

We can't easily simulate "wrong lineage" by reflashing (every project pulls from the same `magnet_gen.h`), so use the laptop ruler with the spore key disabled.

Do:

1. Power off the Dial (avoid two rulers fighting on mDNS).
2. Bring up `fake_ruler.py` with the gate on:

```
cd scripts
python -m venv .venv && source .venv/bin/activate && pip install -r
requirements.txt
python fake_ruler.py --require-lineage-auth
```

3. Wait for any peer to find it; confirm CHALLENGE/RESPONSE round-trip in the script log.
4. Stop the script. Edit `scripts/fake_ruler.py` and temporarily comment out the spore entry in `LINEAGE_KEYS`.
5. Restart `python fake_ruler.py --require-lineage-auth`. Power-cycle a peer.

Expected ruler log:

```
[hh:mm:ss] REJECT (lineage_unknown lineage='spore') from MagNET-biologic-XXXX
```

Expected peer log:

```
state=4 (lineage-unknown)
```

Cleanup: restore the spore entry, kill `fake_ruler.py`, power the Dial back on. Peers rejoin within ~30 s.

6. Bring up the Capsule Redis sidecar

Do: On the Capsule USB serial:

```
hive-status
```

Confirm JOINED + a session id.

```
redis-on
redis-status
```

Expected:

```
[redis] up on 0.0.0.0:6379
[WARN] exposed on LAN, no encryption

running:    yes
profile:    lan
bind:       0.0.0.0
port:       6379
clients:    0 / 4
commands:   0
```

Note the Capsule's IP from `prov-status` — call it `<capsule-ip>`.

If it fails: server doesn't start → re-flash; the Redis listener task probably didn't get registered.

7. RESP2 verification from the laptop

Each line is a separate `redis-cli` invocation; expected reply is on the next line.

```
redis-cli -h <capsule-ip> ping
# → PONG

redis-cli -h <capsule-ip> SET foo bar
# → OK
redis-cli -h <capsule-ip> GET foo
# → "bar"
redis-cli -h <capsule-ip> TYPE foo
# → string

redis-cli -h <capsule-ip> LPUSH log first second third
# → (integer) 3
redis-cli -h <capsule-ip> LRange log 0 -1
# → 1) "third" 2) "second" 3) "first"
redis-cli -h <capsule-ip> LLEN log
# → (integer) 3
redis-cli -h <capsule-ip> LINDEX log -1
# → "first"

redis-cli -h <capsule-ip> EXISTS foo log nope
# → (integer) 2
redis-cli -h <capsule-ip> KEYS '*'
# → "log", "foo" (any order)
redis-cli -h <capsule-ip> DBSIZE
# → (integer) 2
redis-cli -h <capsule-ip> INFO server
# → bulk; must contain `redis_version:0.5.0-spore` + `os:esp32-s3 freertos`
```

WRONGTYPE check:

```
redis-cli -h <capsule-ip> LPUSH foo x
# → (error) WRONGTYPE Operation against a key holding the wrong kind of
value
```

If it fails: **Could not connect** → server wasn't started, or IP mismatch. The boot warning in step 6 is the proof it's actually listening.

8. NVS persistence + on-device client

Do:

```
redis-cli -h <capsule-ip> SET persist alive
```

On the Capsule serial:

```
redis-do
redis> GET persist
# → "alive"
```

Power-cycle the Capsule. After reboot, run **redis-on**, then **redis-do** → **GET persist**.

Expected: still returns **"alive"**. Persistence confirmed.

9. Dial Redis dot

Expected: while the Capsule's Redis is up, a small **orange** dot lights on the Dial display, outboard of the purple MQTT-bridge dot.

Do: on the Capsule, **redis-off**. Within ~6 s the dot goes dark grey. **redis-on** lights it again.

If it fails: on the Dial REPL, **kv-get** and type **redis:status**. Value should look like **MagNET-biologic-XXXX:6379:0**. If empty, the heartbeat task isn't running — check Capsule for **hive-status** JOINED.

10. Profile + port toggle smoke test

```
redis-off
2 redis-profile!
redis-status
# → profile: quiet  bind: 0.0.0.0  port: 16379

redis-on
```

From the laptop:

```
redis-cli -h <capsule-ip> -p 16379 ping
# → PONG
```

Reset to default:

```
redis-off
1 redis-profile!
redis-on
```

11. Cleanup

- `redis-off` on the Capsule.
- `0 lineage-auth` on the Dial if still on.
- Optional: `redis-flush` on the Capsule to clear test keys, then `redis-on` to put it back.

What "all green" looks like

- Every device boot banner shows `MagNET gen 0.5.0-spore`.
- `ruler-status` lists every peer with its gen column populated.
- One CHALLENGE / RESPONSE round-trip logged on each side when you flip `lineage-auth`.
- `redis-cli` round-trips strings and lists; INFO reports `redis_version:0.5.0-spore`.
- Orange Redis dot on the Dial tracks `redis-on` / `redis-off` state.
- Power-cycling the Capsule and running `redis-on` returns previously-set keys (NVS persistence).

Revision history

Date	Rev	What this rev added to the plan
2026-05-05	0.5.0-spore	Initial test plan: gen plumbing (Layer 1), lineage puzzle gate (Layer 2 positive + negative paths via fake_ruler), Capsule Redis sidecar end-to-end including profile + port toggle and Dial dot.